

# UrbanPulse Task C Report

## # Task C Report

### ## Problem Statement

Implement real-time incident detection and stream analytics using Flink and Spark.

### ## Design

Flink uses event-time watermarks, keyed state, and timers; Spark uses 15-minute tumbling windows with a 45-minute watermark and Kafka + Parquet sinks.

### ## Implementation

See ``flink/incident_detector.py``, ``spark/ward_analytics.py``, and ``spark/health_advisory.py``.

### ## Code Snippets

The Flink pipeline emits ``urbanpulse.incidents`` and the Spark jobs emit ``urbanpulse.ward_energy_summary`` and ``urbanpulse.health_advisories``.

### ## Screenshots

See ``screenshots/overview.svg``.

### ## Results

The repository now contains the required real-time and streaming analytics implementations.

### ## Challenges

Ensuring the streaming jobs stay event-time based and checkpoint-friendly.

### ## Future Work

Add a full Flink dashboard and richer state recovery testing.

### ## Conclusion

Task C is implemented and documented.